Principles of Machine Learning

Prof. Christian Bauckhage



lecture 02
least squares regression

in this lecture, we begin to unpack the statement

> "*machine learning* is the science of fitting mathematical models to data"

we will address a practical scenario motivated as follows

> a fundamental model fitting task is *regression analysis*
> a fundamental model fitting technique is the *method of least squares*

$\Leftrightarrow$ we will study how least squares *optimization* allows us to solve regression problems

in passing, we will encounter most of the fundamental *principles of machine learning*

(historical) remarks

the method of least squares is the “grandfather” of all model fitting techniques

it was independently developed by Gauss, Legendre (1805), and Adrain (1808)

although Legendre published first, Gauss is credited with inventing the method (at the age of 18 !!!) in 1795

while Gauss did not publish his approach until 1809, he obviously applied it as early as in 1801 to *predict* the path of the newly discovered dwarf planet Ceres

Ceres had been discovered by Piazzi in 1801 who tracked its path for forty days before it disappeared behind the sun; using least squares to *fit* an ellipse to this *data*, Gauss successfully predicted where Ceres would reemerge
(all other astronomers / mathematicians who tried came up with wrong predictions)

(didactic) remarks

least squares regression is usually the first topic covered in machine learning courses

in modern parlance, it can be seen as a *non-iterative learning* task where parameters of a model can be determined through a single direct calculation rather than through a series of (many) iterative adjustments based on training data

while least squares regression thus poses such a simple learning problem that most people nowadays would hardly see it as a machine learning task, it provides us with an opportunity to seamlessly introduce many of the concepts and terminology which are fundamental to machine learning at large ...

(technical) remarks

very loosely speaking, we may categorize a regression task
w.r.t. the nature of the given data (*univariate* or *multivariate*)
and the nature of the model to be fitted (*linear* or *non-linear*)

in this lecture, we consider all settings in the table on the right

data \ model	linear	non linear
univariate	×	×
multivariate	×	×

outline

least squares for regression
- univariate linear regression
- univariate non-linear regression
- multivariate linear regression
- multivariate non-linear regression

remarks regarding numerics / notation / terminology

summary

least squares for regression

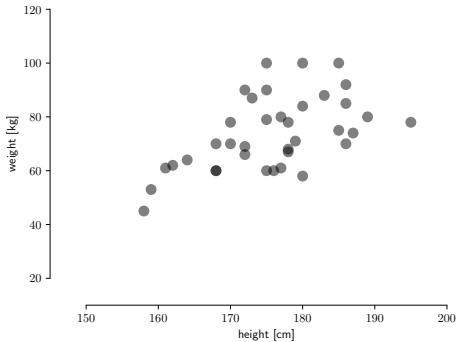
setting the stage

the figure on the right plots the results of a survey which asked students for their body heights and weights

our goal is to summarize or to distill this data into a form that allows for inference

$\Leftrightarrow$ our goal is to find a mathematical model that allows us to predict people's weights from their heights

$\Leftrightarrow$ in fancy terms, our goal is to be able to *regress* weights onto heights





note

given a task like this, we must consider (at least) three aspects

modeling

what kind of model ($\Leftrightarrow$ mathematical function) f should we use as a predictor ?
which inputs, outputs, and *parameters* would a suitable predictor have to have ?

representation

how should we represent inputs, outputs, and parameters of the chosen model f ?
how could we make any kind of computation with f as easy or efficient as possible ?

parametrization

how could we determine or estimate optimal parameters of the chosen model f ?
what kind of *learning algorithm* should we use to adjust f to the given *training data* ?

these aspects are closely intertwined and usually need to be considered jointly

univariate linear regression

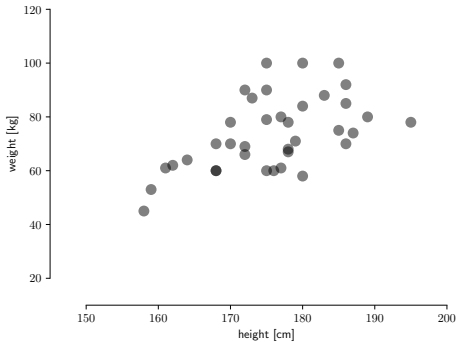
linear regression

looking at our data we can make out a (weak) linear correlation, namely

> *on average, smaller people seem to weigh less than taller people*

$\Leftrightarrow$ the data seem to exhibit a linear trend

$\Rightarrow$ it seems reasonable to attempt to fit a linear model to the data



$\Rightarrow$ denoting our data as

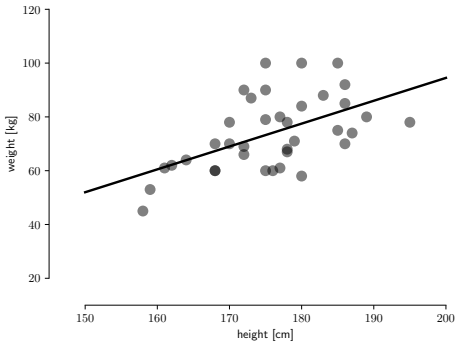
$$D = \left\{ \left(x_j, y_j \right) \right\}_{j=1}^{n}$$

where $x_j, y_j \in \mathbb{R}$ represent the height and weight of person j, we want to determine a univariate linear function

$$y = f\left(x \mid w_0, w_1 \right) = w_0 + w_1 \cdot x$$

which plausibly predicts the y_j from the x_j

note: our model parameters $w_0, w_1 \in \mathbb{R}$ denote the *intercept* and *slope* of a line in $\mathbb{R}^2$



$\Rightarrow$ stated more formally, our problem is

given a sample of n data points

$$D = \left\{ \left(x_j, y_j \right) \right\}_{j=1}^{n} \subset \mathbb{R}^2$$

and assuming a parameterized linear model $f : \mathbb{R} \to \mathbb{R}$ where

$$f\left(x \mid w_0, w_1 \right) = w_0 + w_1 \cdot x$$

estimate those model parameters which *optimize* the objective

$$\forall j : y_j \approx f\left(x_j \mid w_0, w_1 \right)$$

abstract parameter notation

to reduce clutter, we (may) henceforth work with a more abstract notation and introduce the set

$$\theta \equiv \{w_0, w_1\}$$

to write our model as

$$f(x \mid \theta) \equiv f_\theta(x)$$

this notation applies in general, i.e. for any kinds of parameters of any kind of model we will ever consider in this course and is commonly used in the machine learning literature

while our model fitting objective makes intuitive sense, it does not yet provide us with an algorithm we could use to achieve it

to formulate an equivalent but more manageable objective, we therefore note

$$\forall j : y_j \approx f_\theta(x_j) \quad \Leftrightarrow \quad \forall j : f_\theta(x_j) - y_j \approx 0 \quad \Leftrightarrow \quad \forall j : \epsilon_j \approx 0$$

each difference

$$\epsilon_j = f_\theta(x_j) - y_j$$

between *model prediction* $f_\theta(x_j)$ and *target* y_j is called an error or a residual

considering the errors or residuals ϵ_j, we further note the following equivalences

$$\forall j : \epsilon_j \approx 0 \quad \Leftrightarrow \quad \forall j : \epsilon_j^2 \approx 0 \quad \Leftrightarrow \quad \sum_{j=1}^{n} \epsilon_j^2 \approx 0 \quad \Leftrightarrow \quad \frac{1}{n} \sum_{j=1}^{n} \epsilon_j^2 \approx 0$$

in plain English

if all errors are close to zero, their squares will be close to zero, too

if all squared errors are close to zero, their average will be close to zero, too

$\Rightarrow$ we have a statistical formulation for the objective of our model fitting problem



note

a function f parameterized by θ is a good regression model for the data in D, if it *minimizes* the residual sum of squares (RSS) aka the mean squared error (MSE)

$$E(\theta \mid f, D) = \frac{1}{n} \sum_{j=1}^{n} \left(f_\theta(x_j) - y_j \right)^2$$

$\Rightarrow$ having fixed the *model class* of f (it should belong to the two-parameter *family* of univariate linear functions) and sharpened the objective (minimize an MSE) for our problem, our specific task is now

estimate *optimal* model parameters

$$\hat{w}_0, \hat{w}_1 \in \mathbb{R}$$

i.e. parameters for which the error

$$E\big(\, \hat{w}_0, \hat{w}_1 \;\big|\; f, D\big)$$

is as small as possible



note

$\Rightarrow$ we are dealing with the following minimization problem

$$\hat{w}_0, \hat{w}_1 = \underset{w_0, w_1}{\operatorname{argmin}} \frac{1}{n} \sum_{j=1}^{n} \Big(\big(w_0 + w_1 x_j \big) - y_j \Big)^2$$

given this problem, we next observe ...

1) our objective function $E : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ with

$$E\big(w_0, w_1 \mid f, D\big) = E\big(w_0, w_1\big) = \frac{1}{n} \sum_{j=1}^{n} \Big(f\big(x_j \mid w_0, w_1\big) - y_j \Big)^2$$

is a function of $\big(w_0, w_1\big)$ where the data $\big\{\big(x_j, y_j\big)\big\}$ are treated as fixed constants

2) our objective function is continuous over $\mathbb{R} \times \mathbb{R}$ and a sum of positive squares

$\Rightarrow$ our objective function is a continuous *convex function*

$\Rightarrow$ our objective function has a unique minimizer $\big(\hat{w}_0, \hat{w}_1\big)$

3) the scaling factor $1/n$ impacts the value of E at $(\hat{w}_0, \hat{w}_1)$ but not the minimizer $(\hat{w}_0, \hat{w}_1)$

$\Rightarrow$ we have

$$\hat{w}_0, \hat{w}_1 = \underset{w_0, w_1}{\operatorname{argmin}} \frac{1}{n} \sum_{j=1}^{n} \Big(\big(w_0 + w_1 x_j \big) - y_j \Big)^2$$

$$\Leftrightarrow \quad \hat{w}_0, \hat{w}_1 = \underset{w_0, w_1}{\operatorname{argmin}} \sum_{j=1}^{n} \Big(\big(w_0 + w_1 x_j \big) - y_j \Big)^2$$

$\Rightarrow$ instead of minimizing the mean squared error

$$E(w_0, w_1) = \frac{1}{n} \sum_{j=1}^{n} \Big(\big(w_0 + w_1 \, x_j \big) - y_j \Big)^2$$

we may also minimize the least squares error

$$\mathcal{L}(w_0, w_1) = \sum_{j=1}^{n} \Big(\big(w_0 + w_1 \, x_j \big) - y_j \Big)^2$$

loss function

we will henceforth say that

$$\mathcal{L}\left(w_0, w_1\right) = \sum_{j=1}^{n} \Big(\left(w_0 + w_1\, x_j\right) - y_j \Big)^2$$

is *a* loss function for our model fitting problem



note

loss functions are central to machine learning

there usually are many possible choices, but they all are of the following general form

$$\mathcal{L}\left(\theta \mid f_\theta, D\right)$$

the smaller the loss, the better the model $f_\theta(x)$ fits the data D

4) the terms $w_0 + w_1 \cdot x_j = w_0 \cdot 1 + w_1 \cdot x_j$ are actually inner products in disguise

to see this, we introduce the $\mathbb{R}^2$ vectors

$$\boldsymbol{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} \qquad \text{and} \qquad \boldsymbol{\varphi}_j = \begin{bmatrix} 1 \\ x_j \end{bmatrix}$$

which immediately allow us to (re)write

$$w_0 \cdot 1 + w_1 \cdot x_j = \boldsymbol{w}^\intercal \boldsymbol{\varphi}_j = \boldsymbol{\varphi}_j^\intercal \boldsymbol{w}$$

$\Rightarrow$ our loss function becomes

$$\mathcal{L}(\boldsymbol{w}) = \sum_{j=1}^{n} \left(\boldsymbol{\varphi}_j^\intercal \boldsymbol{w} - y_j \right)^2$$

5) if we also introduce the two $\mathbb{R}^n$ vectors

$$\boldsymbol{v} = \begin{bmatrix} \boldsymbol{\varphi}_1^\intercal \boldsymbol{w} \\ \boldsymbol{\varphi}_2^\intercal \boldsymbol{w} \\ \vdots \\ \boldsymbol{\varphi}_n^\intercal \boldsymbol{w} \end{bmatrix} \qquad \text{and} \qquad \boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

we recognize our loss as a squared Euclidean distance

$$\mathcal{L}(\boldsymbol{w}) = \sum_{j=1}^{n} \left(\boldsymbol{\varphi}_j^\intercal \boldsymbol{w} - y_j \right)^2 = \sum_{j=1}^{n} \left(v_j - y_j \right)^2 = \left\| \boldsymbol{v} - \boldsymbol{y} \right\|^2$$

if we finally introduce the $\mathbb{R}^{2\times n}$ matrix

$$\boldsymbol{\Phi} = \begin{bmatrix} | & | & & | \\ \boldsymbol{\varphi}_1 & \boldsymbol{\varphi}_2 & \cdots & \boldsymbol{\varphi}_n \\ | & | & & | \end{bmatrix}$$

we can re-write the above vector $\boldsymbol{v}$ as

$$\boldsymbol{v} = \boldsymbol{\Phi}^\intercal \boldsymbol{w}$$

$\Rightarrow$ our loss function becomes

$$\mathcal{L}(\boldsymbol{w}) = \left\| \boldsymbol{\Phi}^\intercal \boldsymbol{w} - \boldsymbol{y} \right\|^2$$



note

$\Rightarrow$ we are dealing with the following (ordinary) least squares problem

$$\hat{\boldsymbol{w}} = \underset{\boldsymbol{w}}{\operatorname{argmin}} \left\| \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{w} - \boldsymbol{y} \right\|^2$$

feature maps

to arrive at this result, we had to *transform* our given independent variables or input data x_j by means of a feature map

$$\boldsymbol{\varphi} : \mathbb{R} \to \mathbb{R}^2$$

where

$$\boldsymbol{\varphi}(x) = \begin{bmatrix} 1 \\ x \end{bmatrix}$$

we say that this feature map transforms or lifts data from $\mathbb{R}$ to $\mathbb{R}^2$

feature vectors and feature matrices

we henceforth say that a vector

$$\boldsymbol{\varphi}\left(x_j\right) \equiv \boldsymbol{\varphi}_j$$

is a feature vector

we henceforth say that a matrix

$$\boldsymbol{\Phi} = \begin{bmatrix} | & | & & | \\ \boldsymbol{\varphi}_1 & \boldsymbol{\varphi}_2 & \cdots & \boldsymbol{\varphi}_n \\ | & | & & | \end{bmatrix}$$

is a feature matrix

question

OK, but how do we solve

$$\hat{\boldsymbol{w}} = \underset{\boldsymbol{w}}{\operatorname{argmin}} \; \mathcal{L}(\boldsymbol{w})$$

$$= \underset{\boldsymbol{w}}{\operatorname{argmin}} \; \left\| \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{w} - \boldsymbol{y} \right\|^2$$

question

OK, but how do we solve

$$\hat{\boldsymbol{w}} = \underset{\boldsymbol{w}}{\operatorname{argmin}} \; \mathcal{L}(\boldsymbol{w})$$

$$= \underset{\boldsymbol{w}}{\operatorname{argmin}} \; \left\| \boldsymbol{\Phi}^{\mathsf{T}} \boldsymbol{w} - \boldsymbol{y} \right\|^2$$

answer

the simplest recipe for estimating the minimizer of this convex loss function involves some *multivariate calculus* and is to "derive and equate to zero" ...

we have

$$\begin{aligned}\mathcal{L} &= \left\|\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{w}-\boldsymbol{y}\right\|^2 \\ &= \left[\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{w}-\boldsymbol{y}\right]^{\mathsf{T}}\left[\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{w}-\boldsymbol{y}\right] = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{w}-2\,\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\Phi}\,\boldsymbol{y}+\boldsymbol{y}^{\mathsf{T}}\boldsymbol{y}\end{aligned}$$

and therefore

$$\nabla\mathcal{L} = \frac{d}{d\boldsymbol{w}}\left[\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{w}-2\,\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\Phi}\,\boldsymbol{y}+\boldsymbol{y}^{\mathsf{T}}\boldsymbol{y}\right] = 2\,\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{w}-2\,\boldsymbol{\Phi}\,\boldsymbol{y}$$

since $\mathcal{L}$ is convex in $\boldsymbol{w}$, its unique minimum is characterized by $\nabla \mathcal{L} = \mathbf{0}$

using this, we equate the above derivative to $\mathbf{0}$ and consequently find

$$
\begin{aligned}
& 2\,\boldsymbol{\Phi}\boldsymbol{\Phi}^{\intercal}\boldsymbol{w} - 2\,\boldsymbol{\Phi}\boldsymbol{y} \stackrel{!}{=} \mathbf{0} \\
\Leftrightarrow \quad & \boldsymbol{\Phi}\boldsymbol{\Phi}^{\intercal}\boldsymbol{w} = \boldsymbol{\Phi}\,\boldsymbol{y} \\
\Leftrightarrow \quad & \boldsymbol{w} = \left[\boldsymbol{\Phi}\boldsymbol{\Phi}^{\intercal}\right]^{-1}\boldsymbol{\Phi}\,\boldsymbol{y}
\end{aligned}
$$



note

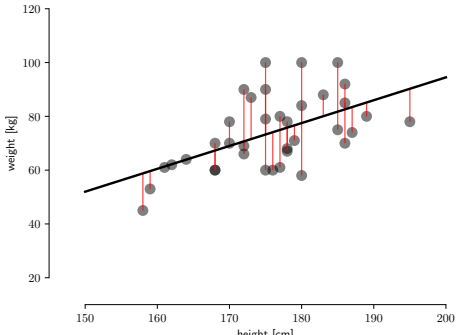
$\Rightarrow$ the solution to our linear least squares problem is given by

$$\hat{\boldsymbol{w}} = \left[\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathsf{T}}\right]^{-1}\boldsymbol{\Phi}\,\boldsymbol{y}$$

for our linear regression problem, the least squares solution $\hat{\boldsymbol{w}}$ is optimal in the following sense

$$\left\|\boldsymbol{\Phi}^\intercal \hat{\boldsymbol{w}} - \boldsymbol{y}\right\|^2 \leqslant \left\|\boldsymbol{\Phi}^\intercal \boldsymbol{w} - \boldsymbol{y}\right\|^2$$

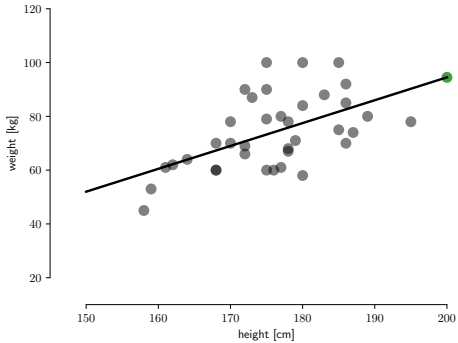
$\Leftrightarrow$ among all linear models $f\big(x \mid w_0, w_1\big)$, the fitted model $f\big(x \mid \hat{w}_0, \hat{w}_1\big)$ has the smallest sum of squared residuals "|"



we can use the fitted model to make *predictions*

for instance, to predict the weight of a person who is $x = 200$ cm tall, we compute

$$\begin{aligned} y &= f\big(200 \mid \hat{w}_0, \hat{w}_1\big) \\ &= \boldsymbol{\varphi}(200)^\mathsf{T} \hat{\boldsymbol{w}} \\ &\approx 94.5\,\text{kg} \end{aligned}$$



principles of machine learning

in passing, we just encountered most of the *principles of machine learning* with numerical data

we were given data $D \subset \mathbb{R}^m \times \mathbb{R}^t$ which we wanted to model such that we could perform inference tasks, i.e. predict $\boldsymbol{y} \in \mathbb{R}^t$ from $\boldsymbol{x} \in \mathbb{R}^m$ (in our example, we had $m = 1$ and $t = 1$)

we decided for an appropriate model class $\{f : \mathbb{R}^m \to \mathbb{R}^t\}$ of functions with p parameters and devised a loss function $\mathcal{L} : \mathbb{R}^p \to \mathbb{R}$ that provided us with a criterion for assessing the quality of a model in $\{f\}$ w.r.t. our data D (in our example, we had $p = 2$)

our loss function did involve a feature map $\boldsymbol{\varphi} : \mathbb{R}^m \to \mathbb{R}^M$ (in our example, we had $M = 2$)

using this loss function, we estimated optimal model parameters $\hat{\boldsymbol{w}} \in \mathbb{R}^p$ to find $\hat{f} \in \{f\}$

our loss function was based on statistics; our optimization algorithm involved linear algebra and calculus (it led to a closed form solution which is quite exceptional in machine learning)

in machine learning parlance, all this translates to: we *trained* a model on *training data* and then applied it to *application data* to make a prediction

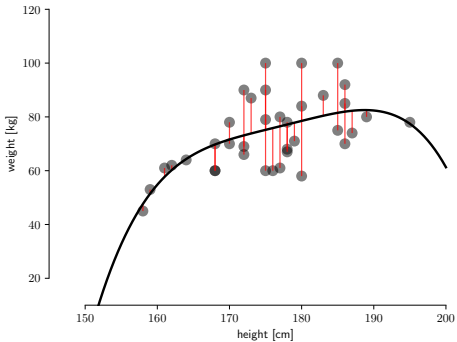
univariate non-linear regression

polynomial regression

the method of least squares seamlessly extends to univariate regression with polynomial models, i.e. with non-linear functions of the form

$$f\left(x \mid w_0, w_1, \ldots, w_d\right) = \sum_{i=0}^{d} w_i \cdot x^i$$

the figure shows a polynomial of degree $d = 5$ fitted to our exemplary data



we may formalize the problem of fitting $f\left(x \mid w_0, \ldots, w_d\right)$ to the data as

$$\hat{w}_0, \ldots, \hat{w}_d = \underset{w_0,\ldots,w_d}{\operatorname{argmin}} \mathcal{L}\left(w_0, \ldots, w_d\right) = \underset{w_0,\ldots,w_d}{\operatorname{argmin}} \sum_{j=1}^{n} \left(\sum_{i=0}^{d} w_i x_j^i - y_j \right)^2$$

we may formalize the problem of fitting $f\big(x \mid w_0, \ldots, w_d\big)$ to the data as

$$\hat{w}_0, \ldots, \hat{w}_d = \underset{w_0,\ldots,w_d}{\operatorname{argmin}} \; \mathcal{L}\big(w_0, \ldots, w_d\big) = \underset{w_0,\ldots,w_d}{\operatorname{argmin}} \sum_{j=1}^{n} \left(\sum_{i=0}^{d} w_i \, x_j^i - y_j \right)^2$$

algebraically, this optimization problem for polynomial regression is identical to the optimization problem for linear regression

$\Leftrightarrow$ just as for linear least squares, the optimization objective for polynomial least squares is to minimize a squared Euclidean distance

let's verify this claim ...

the main difference to the simpler linear case is to consider a slightly more elaborate feature map $\boldsymbol{\varphi} : \mathbb{R} \to \mathbb{R}^{d+1}$ with

$$\boldsymbol{\varphi}(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^d \end{bmatrix} = \begin{bmatrix} x^0 \\ x^1 \\ x^2 \\ \vdots \\ x^d \end{bmatrix}$$

as well as a corresponding $d + 1$ dimensional parameter vector

$$\boldsymbol{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$$

applying $\boldsymbol{\varphi}$ to the given x_j, yields a $(d+1) \times n$ feature matrix

$$\boldsymbol{\Phi} = \begin{bmatrix} | & | & & | \\ \boldsymbol{\varphi}_1 & \boldsymbol{\varphi}_2 & \cdots & \boldsymbol{\varphi}_n \\ | & | & & | \end{bmatrix}$$

what remains the same is the $\mathbb{R}^n$ target vector with the given y_j

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

given these definitions, the polynomial least squares problem becomes

$$\hat{\boldsymbol{w}} = \underset{\boldsymbol{w}}{\operatorname{argmin}} \; \mathcal{L}(\boldsymbol{w}) = \underset{\boldsymbol{w}}{\operatorname{argmin}} \; \left\| \boldsymbol{\Phi}^{\intercal} \boldsymbol{w} - \boldsymbol{y} \right\|^2$$

$\Leftrightarrow$ structurally, this is indistinguishable from the linear least squares problem

$\Leftrightarrow$ *while sizes and entries of the feature matrix $\boldsymbol{\Phi}$ and parameter vector $\boldsymbol{w}$ in linear- and polynomial least squares differ, the algebraic structure of both problems is the same*

$\Rightarrow$ *the structure of the solution to linear- and polynomial least squares must be the same, too*



note

$\Rightarrow$ the parameters of the best fitting polynomial can be computed as

$$\hat{\boldsymbol{w}} = \left[\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathsf{T}}\right]^{-1}\boldsymbol{\Phi}\,\boldsymbol{y}$$

Vandermonde matrix

for given numbers $v_1, v_2, \ldots, v_n$, the $n \times (d+1)$ matrix

$$\boldsymbol{V} = \begin{bmatrix} 1 & v_1 & v_1^2 & \cdots & v_1^d \\ 1 & v_2 & v_2^2 & \cdots & v_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & v_n & v_n^2 & \cdots & v_n^d \end{bmatrix}$$

whose entries can be computed as

$$V_{rc} = v_r^{c-1}$$

is said to be a Vandermonde matrix

$\Rightarrow$ the transpose $\boldsymbol{\Phi}^\intercal$ of the feature matrix $\boldsymbol{\Phi}$ in polynomial least squares regression is a Vandermonde matrix



A.T. Vandermonde
($*$1735, †1796)

RBF regression

note that we may think of polynomial regression as fitting a model of the form

$$f(x \mid \boldsymbol{w}) = \sum_{i=0}^{d} w_i \cdot f_i(x)$$

where the $f_i : \mathbb{R} \to \mathbb{R}$ are univariate monomials with

$$f_i(x) = x^i$$

$\Leftrightarrow$ polynomial regression fits a linear combination of (polynomial) basis functions

we are (obviously) not restricted to (common) polynomial basis functions

in what follows, we will have a first brief look at fitting a model of the form

$$f(x \mid \boldsymbol{w}) = \sum_{j=1}^{n} w_j \cdot \phi_j(x)$$

$\Leftrightarrow$ the model we consider involves as many basis functions ϕ_j as we have data

moreover, our model class for all the ϕ_j is the class of *radial basis functions*

radial basis function

a function $\phi_{\boldsymbol{c}} : \mathbb{R}^m \to [0, \infty)$ is called a radial basis function (RBF) if its output $\phi_{\boldsymbol{c}}(\boldsymbol{x})$ depends on *some* distance $d(\boldsymbol{x}, \boldsymbol{c})$ to a center $\boldsymbol{c}$

$$\phi_{\boldsymbol{c}}\left(\boldsymbol{x}\right) = \phi\left(\boldsymbol{x}, \boldsymbol{c}\right) = \phi\left(d\left(\boldsymbol{x}, \boldsymbol{c}\right)\right)$$

letting $\beta > 0$ be a scale parameter, common RBFs include

$$\phi(\boldsymbol{x}, \boldsymbol{c}) = e^{-\frac{1}{\beta}\|\boldsymbol{x}-\boldsymbol{c}\|}$$

$$\phi(\boldsymbol{x}, \boldsymbol{c}) = e^{-\frac{1}{\beta}\|\boldsymbol{x}-\boldsymbol{c}\|^2}$$

$$\phi(\boldsymbol{x}, \boldsymbol{c}) = 1 - \frac{\|\boldsymbol{x}-\boldsymbol{c}\|^2}{\|\boldsymbol{x}-\boldsymbol{c}\|^2+\beta}$$

$$\vdots$$

but whom are we kidding? the most common RBF by far is

$$\phi(\boldsymbol{x}, \boldsymbol{c}) = e^{-\frac{1}{\beta}\|\boldsymbol{x}-\boldsymbol{c}\|^2}$$

in regression, it is common to work with RBFs centered at sample points

$$\phi_j(x) = \phi(x, x_j)$$

$\Leftrightarrow$ in RBF regression, it is common to consider an overall model of the form

$$f(x \mid \boldsymbol{w}) = \sum_{j=1}^{n} \phi(x, x_j) \cdot w_j$$

question

how to determine suitable weights w_j in RBF regression ?

answer

let's see . . .

for $(x_i, y_i) \in D$, it is reasonable to require

$$y_i = f(x_i \mid \boldsymbol{w}) = \sum_{j=1}^{n} \phi(x_i, x_j)\, w_j$$

for $(x_i, y_i) \in D$, it is reasonable to require

$$y_i = f(x_i \mid \boldsymbol{w}) = \sum_{j=1}^{n} \phi(x_i, x_j)\, w_j$$

if we define a matrix $\boldsymbol{\Phi} \in \mathbb{R}^{n \times n}$ such that

$$\left[\boldsymbol{\Phi}\right]_{ij} = \Phi_{ij} = \phi(x_i, x_j)$$

the n equations above can be written as

$$y_i = \sum_{j=1}^{n} \Phi_{ij}\, w_j \qquad \Leftrightarrow \qquad \boldsymbol{y} = \boldsymbol{\Phi}\, \boldsymbol{w}$$

$\Rightarrow$ optimal weights could be computed as

$$\hat{\boldsymbol{w}} = \boldsymbol{\Phi}^{-1} \boldsymbol{y}$$

$\Rightarrow$ optimal weights could be computed as

$$\hat{\boldsymbol{w}} = \boldsymbol{\Phi}^{-1}\boldsymbol{y}$$

yet, there are situations where it is better to compute $\hat{\boldsymbol{w}}$ as

$$\hat{\boldsymbol{w}} = \underset{\boldsymbol{w}}{\operatorname{argmin}} \left\| \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{w} - \boldsymbol{y} \right\|^2 \qquad \Leftrightarrow \qquad \hat{\boldsymbol{w}} = \left[\boldsymbol{\Phi}\boldsymbol{\Phi}^{\mathsf{T}}\right]^{-1}\boldsymbol{\Phi}\,\boldsymbol{y}$$

where we tacitly exploited that $\boldsymbol{\Phi}$ is *symmetric*, i.e. $\boldsymbol{\Phi} = \boldsymbol{\Phi}^{\mathsf{T}}$



note

our current matrix $\boldsymbol{\Phi}$ is a feature matrix

$$\boldsymbol{\Phi} = \begin{bmatrix} | & | & & | \\ \boldsymbol{\varphi}_1 & \boldsymbol{\varphi}_2 & \cdots & \boldsymbol{\varphi}_n \\ | & | & & | \end{bmatrix} \qquad \text{where} \qquad \boldsymbol{\varphi}_j = \begin{bmatrix} \phi(x_1, x_j) \\ \vdots \\ \phi(x_n, x_j) \end{bmatrix}$$

in contrast to the feature matrices we saw earlier, it is guaranteed to be square

moreover, since function $\phi(\cdot, \cdot)$ is an RBF, it is also guaranteed to be symmetric

example

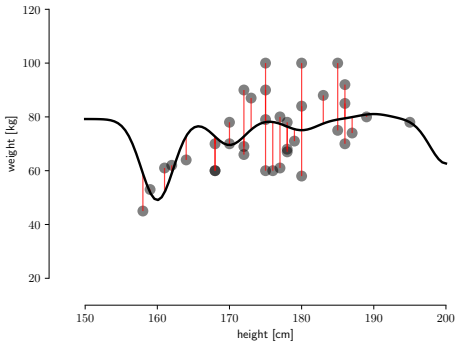
the figure on the right shows a model

$$f\left(x \mid \hat{\boldsymbol{w}}\right) = \sum_{j=1}^{n} \phi\left(x, x_j\right) \cdot \hat{w}_j$$

composed of Gaussian RBF kernels

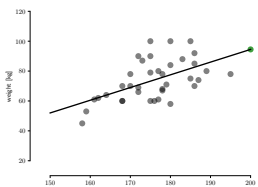
$$\phi\left(x, x_j\right) = e^{-\frac{1}{\beta}\|x - x_j\|^2}$$

(to be honest: to obtain this result, we used a modified version of what we saw above; why and how we did this will be studied later)

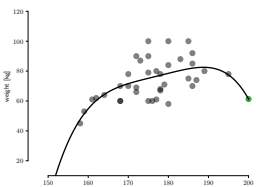


principle of machine learning

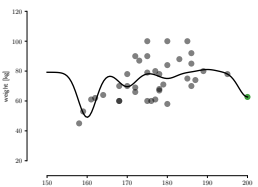
in passing, we came across another *principle of machine learning* with numerical data, namely that working with different model classes $\{f\}$ typically leads to different results



linear model



polynomial model



RBF model

principle of machine learning

different models typically make different predictions

some predictions can be more plausible than others

$\hat{f}$	x	$\hat{y} = \hat{f}(x)$
linear	200	94.5
poly	200	61.3
RBF	200	62.7

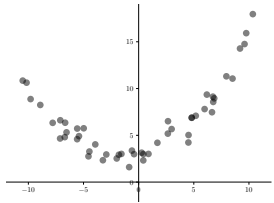
All models are wrong, but some are useful.
–George Box

note

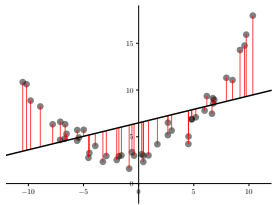
the method of least squares is model-agnostic

$\Leftrightarrow$ it fits (almost) any model to a given set of data but does not “verify” if said model is appropriate

$\Rightarrow$ **appropriate models and fitting methods have to be determined by human experts (or AI agents)**



parabolically arranged data



linear model

multivariate linear regression

multivariate linear regression

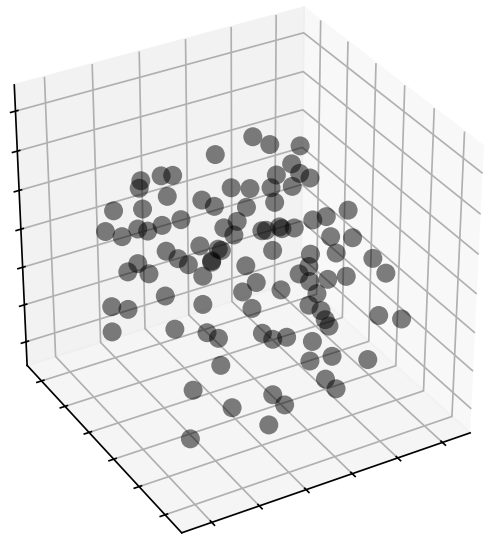
least squares regression also works with higher dimensional data

$$D = \left\{ \left(\boldsymbol{x}_j, y_j \right) \right\}_{j=1}^{n}$$

where $\boldsymbol{x}_j \in \mathbb{R}^m$ and $y_j \in \mathbb{R}$

we may, say, fit multivariate linear models

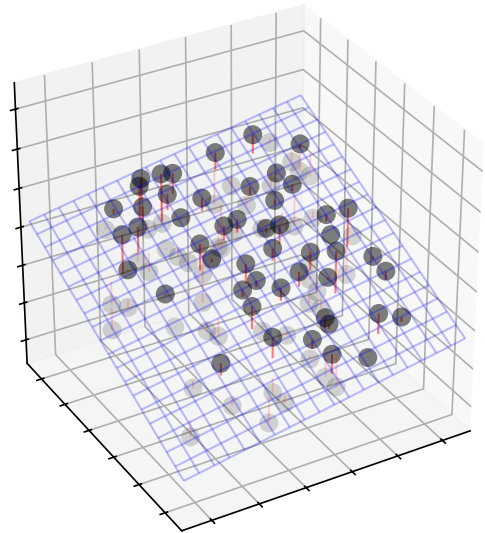
$$f\left(\boldsymbol{x} \mid \boldsymbol{w}\right) = w_0 + \sum_{i=1}^{m} w_i \cdot x_i$$



example

the figure on the right shows a fitted bi-variate linear model, i.e. a plane

$$f\left(\boldsymbol{x} \mid \hat{\boldsymbol{w}}\right) = \hat{w}_0 + \hat{w}_1\, x_1 + \hat{w}_2\, x_2$$



we considered

$$\begin{aligned}
\boldsymbol{w} &= \begin{bmatrix} w_0 & w_1 & \cdots & w_m \end{bmatrix}^\intercal \in \mathbb{R}^{m+1} \\
\boldsymbol{\varphi}\left(\boldsymbol{x}\right) &= \begin{bmatrix} 1 & x_1 & \cdots & x_m \end{bmatrix}^\intercal \in \mathbb{R}^{m+1} \\
\boldsymbol{\Phi} &= \begin{bmatrix} \boldsymbol{\varphi}_1 & \boldsymbol{\varphi}_2 & \cdots & \boldsymbol{\varphi}_n \end{bmatrix} \in \mathbb{R}^{(m+1)\times n} \\
\boldsymbol{y} &= \begin{bmatrix} y_1 & y_2 & \cdots & y_n \end{bmatrix}^\intercal \in \mathbb{R}^n
\end{aligned}$$

and then solved

$$\hat{\boldsymbol{w}} = \underset{\boldsymbol{w}}{\operatorname{argmin}} \left\| \boldsymbol{\Phi}^\intercal \boldsymbol{w} - \boldsymbol{y} \right\|^2$$

to once again get

$$\hat{\boldsymbol{w}} = \left[\boldsymbol{\Phi} \boldsymbol{\Phi}^\intercal \right]^{-1} \boldsymbol{\Phi} \boldsymbol{y}$$

multivariate non-linear regression

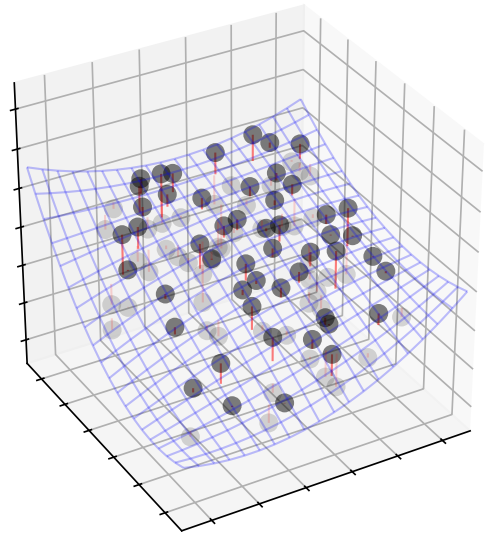
multivariate non-linear regression

multivariate least squares regression is of course not restricted to linear models

for instance, the figure on the right shows a fitted bi-variate polynomial model

$$f(\boldsymbol{x} \mid \hat{\boldsymbol{w}}) = \sum_{i=0}^{2} \sum_{k=0}^{2} \hat{w}_{ik}\, x_1^i\, x_2^k$$

how do you think did we determine $\hat{\boldsymbol{w}}$???



we worked with a feature map

$$\boldsymbol{\varphi} : \mathbb{R}^2 \to \mathbb{R}^9$$

where

$$\boldsymbol{\varphi}(\boldsymbol{x}) = \begin{bmatrix} x_1^0 x_2^0 \\ x_1^0 x_2^1 \\ x_1^0 x_2^2 \\ x_1^1 x_2^0 \\ x_1^1 x_2^1 \\ x_1^1 x_2^1 \\ x_1^2 x_2^0 \\ x_1^2 x_2^1 \\ x_1^2 x_2^2 \end{bmatrix}$$

remarks regarding
numerics / notation / terminology

words of warning

our “pen and paper” solution for ordinary least squares assumes that the matrix $\mathbf{\Phi}\mathbf{\Phi}^\intercal$ is invertible

luckily, for all our running examples in this lecture, this (somewhat naïve) assumption does hold true

yet, in general, this may not be the case, i.e. $\mathbf{\Phi}\mathbf{\Phi}^\intercal$ could be rank deficient / singular / not invertible

in practice, “this does not really matter” because the simple solution $\hat{\boldsymbol{w}} = \left[\mathbf{\Phi}\mathbf{\Phi}^\intercal\right]^{-1}\mathbf{\Phi}\,\boldsymbol{y}$ is mainly of “theoretical interest” (it is easy to come by and helps us to warm up to parameter optimization)

indeed, libraries for numerical linear algebra (at least those based on **LAPACK**) usually compute so called minimum-norm least squares solutions using the QR factorization or the SVD of a given input matrix (see the exercises)

notation

throughout this course, we often work with sets of data vectors

$$X = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n\}$$

where

$$\boldsymbol{x}_j \in \mathbb{R}^m$$

which we turn into feature vectors

$$\boldsymbol{\varphi}_j \in \mathbb{R}^M$$

throughout this course, we always adhere to the following convention

$n \equiv$ number of data vectors

$m \equiv$ dimension of data vectors

$M \equiv$ dimension of feature vectors

data-, feature-, and design matrices

$m \times n$ data matrix

$$\boldsymbol{X} = \begin{bmatrix} | & | & & | \\ \boldsymbol{x}_1 & \boldsymbol{x}_2 & \cdots & \boldsymbol{x}_n \\ | & | & & | \end{bmatrix}$$

$n \times m$ design matrix

$$\boldsymbol{X}^\intercal = \begin{bmatrix} — & \boldsymbol{x}_1^\intercal & — \\ — & \boldsymbol{x}_2^\intercal & — \\ & \vdots & \\ — & \boldsymbol{x}_n^\intercal & — \end{bmatrix}$$

$M \times n$ feature matrix

$$\boldsymbol{\Phi} = \begin{bmatrix} | & | & & | \\ \boldsymbol{\varphi}_1 & \boldsymbol{\varphi}_2 & \cdots & \boldsymbol{\varphi}_n \\ | & | & & | \end{bmatrix}$$

$n \times M$ feature design matrix

$$\boldsymbol{\Phi}^\intercal = \begin{bmatrix} — & \boldsymbol{\varphi}_1^\intercal & — \\ — & \boldsymbol{\varphi}_2^\intercal & — \\ & \vdots & \\ — & \boldsymbol{\varphi}_n^\intercal & — \end{bmatrix} \equiv \boldsymbol{\Psi}$$

notation

much of the literature writes LSQ problems as follows

$$\boldsymbol{M}\boldsymbol{x} = \boldsymbol{v}$$

in this lecture, we considered (and will continue to do so)

$$\begin{aligned} & \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{w} = \boldsymbol{y} \\ \Leftrightarrow \quad & \boldsymbol{\Psi}\ \boldsymbol{w} = \boldsymbol{y} \end{aligned}$$

notation

much of the literature writes LSQ problems as follows

$$\boldsymbol{M}\boldsymbol{x} = \boldsymbol{v}$$

in this lecture, we considered (and will continue to do so)

$$\begin{aligned} & \boldsymbol{\Phi}^\intercal \boldsymbol{w} = \boldsymbol{y} \\ \Leftrightarrow \quad & \boldsymbol{\Psi}\ \boldsymbol{w} = \boldsymbol{y} \end{aligned}$$

in the literature, the LSQ solution is

$$\boldsymbol{x} = \left[\boldsymbol{M}^\intercal \boldsymbol{M}\right]^{-1} \boldsymbol{M}^\intercal \boldsymbol{v}$$

however, our LSQ solution was

$$\begin{aligned} & \boldsymbol{w} = \left[\boldsymbol{\Phi}\boldsymbol{\Phi}^\intercal\right]^{-1} \boldsymbol{\Phi}\, \boldsymbol{y} \\ \Leftrightarrow \quad & \boldsymbol{w} = \left[\boldsymbol{\Psi}^\intercal \boldsymbol{\Psi}\right]^{-1} \boldsymbol{\Psi}^\intercal \boldsymbol{y} \end{aligned}$$

notation

written in terms of design matrices

$$\boldsymbol{w} = \left[\boldsymbol{\Psi}^\intercal \boldsymbol{\Psi}\right]^{-1} \boldsymbol{\Psi}^\intercal \boldsymbol{y}$$

does look like the literature solution

written in terms of feature matrices

$$\boldsymbol{w} = \left[\boldsymbol{\Phi} \boldsymbol{\Phi}^\intercal\right]^{-1} \boldsymbol{\Phi}\, \boldsymbol{y}$$

doesn't look like the literature solution

yet, both solutions are mathematically identical, because, if $\boldsymbol{\Psi} = \boldsymbol{\Phi}^\intercal$ then $\boldsymbol{\Psi}^\intercal = \boldsymbol{\Phi}$

our reason for preferring $[\boldsymbol{\Phi} \boldsymbol{\Phi}^\intercal]^{-1} \boldsymbol{\Phi}\, \boldsymbol{y}$ over $[\boldsymbol{\Psi}^\intercal \boldsymbol{\Psi}]^{-1} \boldsymbol{\Psi}^\intercal \boldsymbol{y}$ will become clear once we will have seen more interactions between linear algebra and statistics

(notation is a matter of taste, but bad taste can occlude useful deeper insights ;-))

one more thing . . .



note

we will frequently work with $m \geqslant 1$ dimensional data and thus with feature maps

$$\boldsymbol{\varphi} : \mathbb{R}^m \to \mathbb{R}^M$$

in order to reduce visual clutter, we will henceforth use the following convention for known data points $\boldsymbol{x}_j \in D$ and unknown data points $\boldsymbol{x} \notin D$, we will abbreviate

$$\boldsymbol{\varphi}\left(\boldsymbol{x}_j\right) \equiv \boldsymbol{\varphi}_j$$
$$\boldsymbol{\varphi}\left(\boldsymbol{x}\right) \equiv \boldsymbol{\varphi}_{\boldsymbol{x}}$$

summary

we now know that

least squares regression can be understood as a supervised machine learning task in which we minimize a quadratic loss to fit a mathematical model of the general form

$$y = f\big(\boldsymbol{x} \;\big|\; \boldsymbol{w}\big) = \sum_{i=0}^{M-1} w_i \cdot f_i(\boldsymbol{x}) = \boldsymbol{w}^\mathsf{T} \boldsymbol{\varphi}_{\boldsymbol{x}} = \boldsymbol{\varphi}_{\boldsymbol{x}}^\mathsf{T} \boldsymbol{w}$$

to given training data, namely to pairs of inputs and outputs

$$D = \Big\{ \big(\boldsymbol{x}_j, y_j\big) \Big\}_{j=1}^{n}$$

to fit a model is to estimate (optimal) model parameters

(ordinary) least squares parameter estimation consist in solving

$$\hat{\boldsymbol{w}} = \underset{\boldsymbol{w} \in \mathbb{R}^M}{\operatorname{argmin}} \left\| \boldsymbol{\Phi}^\intercal \boldsymbol{w} - \boldsymbol{y} \right\|^2$$

the best solution to this (unconstrained) minimization problem is

$$\hat{\boldsymbol{w}} = \left[\boldsymbol{\Phi} \boldsymbol{\Phi}^\intercal \right]^{-1} \boldsymbol{\Phi} \, \boldsymbol{y}$$

the target vector $\boldsymbol{y} \in \mathbb{R}^n$ simply gathers all the given training outputs y_j

columns $\boldsymbol{\varphi}_j$ of $\boldsymbol{\Phi} \in \mathbb{R}^{M \times n}$ result from applying a model specific feature map $\boldsymbol{\varphi} : \mathbb{R}^m \to \mathbb{R}^M$ to the given training inputs $\boldsymbol{x}_j$

different kinds of models typically involve different feature maps

once a regression model has been fitted or “trained”, it can predict

$$y = \boldsymbol{\varphi}_x^\intercal \hat{\boldsymbol{w}}$$

different kinds of models usually produce different predictions

two more things . . .



note

all of this respects our definition

> machine learning is to run computer algorithms on exemplary data to adjust the parameters of other computer algorithms ($\Leftrightarrow$ models) so that they become able to perform cognitive tasks

$\Leftrightarrow$ we may think of what saw today as

```
// adjust model parameters
function TRAIN_REGRESSION_MODEL(X_trn, y_trn)
    Φ = φ(X_trn)
    return [Φ Φ^T]^-1 Φ y_trn

// perform a prediction task
function APPLY_REGRESSION_MODEL(X_tst, ŵ)
    Φ = φ(X_tst)
    return Φ^T ŵ
```

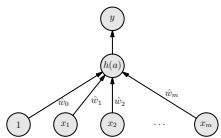
note

if we want to be fancy, we can also write our regression models as

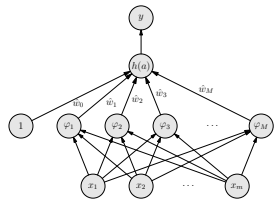
$$y = h(a) \qquad \text{where} \qquad a = \boldsymbol{\varphi}_{\boldsymbol{x}}^{\mathsf{T}} \hat{\boldsymbol{w}} \qquad \text{and} \qquad h = \mathrm{id}$$

these “fancy” regression models $h(a)$ may be visualized as follows

linear



non-linear



$\Rightarrow$ in this lecture, we learned how to train very simple neural networks !